

Movie Rental Data Warehouse Design

Section 1: Introduction

This report proposes a data warehouse design for a movie rental business built on top of an existing OLTP database. The OLTP system handles daily operations: registering customers, managing films and inventory, recording rentals, processing payments, and maintaining staff and store data.

The OLTP system is optimized for fast transactional processing, not for reporting. A data warehouse reorganizes that operational data into a structure built for analytical queries, historical trend analysis, and business intelligence.

OLTP vs Data Warehouse

Feature	OLTP (Source)	Data Warehouse (Target)
Purpose	Day-to-day transaction processing	Analytical reporting and decision-making
Data Type	Current operational data	Historical aggregated data
Schema	Normalized (3NF)	Denormalized (Star Schema)
Operations	INSERT, UPDATE, DELETE (write-heavy)	SELECT with aggregations (read-heavy)
Performance	Optimized for small fast transactions	Optimized for complex large queries
Example Tables	rental, payment, inventory	Fact_Rental, Dim_Customer, Dim_Film

Section 2: Business Questions

The following 10 business questions guide the warehouse design. They cover the main analytical needs of the rental business.

2.1 Rental Analysis Questions

1. Which films are rented most frequently?
2. Which film categories are most popular?
3. Which rentals were returned late most often?
4. How does rental activity change over time?
5. Which stores process the highest number of rentals?

2.2 Payment and Revenue Analysis Questions

6. Which films generate the highest revenue?
7. Which customers generate the highest revenue?
8. How does revenue change over time?

2.3 Inventory Analysis Questions

9. How many inventory copies are available for each film and store?

2.4 Overall Business Performance Question

10. Which stores perform best in terms of rentals, inventory, and revenue?

Section 3: Dimensional Model Design

3.1 Selected Business Processes

Three business processes were identified:

- Film Rental Transactions: every time a customer rents a film. This is the core event.
- Payment Transactions: every time a payment is recorded against a rental.
- Inventory Stock: each physical film copy available in a store.

Together these three processes cover film activity, inventory availability, customer behavior, store operations, and all financial reporting.

3.2 Fact Tables

Fact Table 1: Fact_Rental

Property	Description
Business Process	Film rental transaction — one event per rental
Grain	One row per individual rental transaction
Source OLTP Tables	rental, inventory, film, customer, staff, store
Schema Type	Star Schema

Measures:

Column	Data Type	Description
rental_count	INTEGER	Always 1; used for COUNT aggregations
rental_duration_days	INTEGER	Actual days kept (return_date – rental_date)
expected_duration_days	INTEGER	Maximum allowed days from film table
late_return_flag	BOOLEAN	1 if rental_duration_days > expected_duration_days
days_overdue	INTEGER	Days past due (0 if returned on time)

Foreign Keys:

Foreign Key	References
rental_id	Original rental ID from OLTP system
rental_date_key	Dim_Date

Foreign Key	References
return_date_key	Dim_Date (second role-playing FK)
customer_key	Dim_Customer
film_key	Dim_Film
store_key	Dim_Store
staff_key	Dim_Staff

Fact Table 2: Fact_Payment

Property	Description
Business Process	Payment transaction — one event per payment
Grain	One row per individual payment transaction
Source OLTP Tables	payment, rental, customer, staff, inventory
Schema Type	Star Schema

Measures:

Column	Data Type	Description
payment_amount	DECIMAL(10,2)	Amount paid by the customer
payment_count	INTEGER	Always 1; used for COUNT aggregations

Foreign Keys and Degenerate Dimension:

Column	Type	Description
payment_date_key	FK → Dim_Date	Date the payment was made
customer_key	FK → Dim_Customer	Customer who made the payment
staff_key	FK → Dim_Staff	Staff member who processed the payment
rental_id	INTEGER	Original rental reference from the OLTP system.
film_key	FK → Dim_Film	Film related to the payment
store_key	FK → Dim_Store	Store related to the payment

Fact Table 3: Fact_Inventory

Property	Description
Business Process	Inventory stock
Grain	One row per physical inventory item / film copy
Source OLTP Tables	inventory, film, store

Property	Description
Schema Type	Star Schema

Measures:

Column	Data Type	Description
inventory_count	INTEGER	Always 1; used to count physical film copies

Foreign Keys:

Column	Type	Description
inventory_date_key	FK → Dim_Date	Date the inventory item was last updated
film_key	FK → Dim_Film	Film related to the inventory item
store_key	FK → Dim_Store	Store where the inventory item is available
inventory_id	INTEGER	Original inventory reference from the OLTP system

3.3 Dimension Tables

Dim_Date

Purpose: time-based analysis across three fact tables. Shared dimension.

Column	Data Type	Description
date_key (PK)	INTEGER	YYYYMMDD format, e.g. 20240215
full_date	DATE	Full date value
day_of_week	VARCHAR(10)	Monday, Tuesday, etc.
day_of_month	INTEGER	1 to 31
month_number	INTEGER	1 to 12
month_name	VARCHAR(10)	January, February, etc.
quarter	INTEGER	1 to 4
year	INTEGER	e.g. 2024
is_weekend	BOOLEAN	1 if Saturday or Sunday
season	VARCHAR(10)	Spring, Summer, Autumn, Winter

Dim_Customer

Purpose: customer behavior, segmentation, location reporting. Source: customer, address, city, country.

Column	Data Type	Description
customer_key (PK)	INTEGER	Surrogate key

Column	Data Type	Description
customer_id	INTEGER	Original OLTP ID (natural key)
full_name	VARCHAR(100)	First + last name concatenated
email	VARCHAR(100)	Customer email
city	VARCHAR(50)	City of residence
country	VARCHAR(50)	Country of residence
postal_code	VARCHAR(15)	Postal code
active_status	VARCHAR(10)	Active or Inactive
create_date	DATE	Registration date

Dim_Film

Purpose: film popularity, category analysis, revenue per film. Source: film, language, film_category, category.

Column	Data Type	Description
film_key (PK)	INTEGER	Surrogate key
film_id	INTEGER	Original OLTP ID (natural key)
title	VARCHAR(255)	Film title
description	TEXT	Film description
release_year	INTEGER	Year of release
language	VARCHAR(50)	Film language
rental_duration	INTEGER	Allowed rental days
rental_rate	DECIMAL(5,2)	Standard rental price
length_minutes	INTEGER	Film length in minutes
rating	VARCHAR(10)	G, PG, PG-13, R, NC-17
category	VARCHAR(50)	Action, Comedy, Drama, etc.
special_features	VARCHAR(255)	Behind the Scenes, Trailers, etc.

Dim_Store

Purpose: store-level performance and location reporting. Source: store, address, city, country.

Column	Data Type	Description
store_key (PK)	INTEGER	Surrogate key
store_id	INTEGER	Original OLTP ID
store_address	VARCHAR(100)	Full street address
city	VARCHAR(50)	City
country	VARCHAR(50)	Country

Column	Data Type	Description
postal_code	VARCHAR(15)	Postal code

Dim_Staff

Purpose: staff performance tracking. Source: staff, address, city, country, store.

Column	Data Type	Description
staff_key (PK)	INTEGER	Surrogate key
staff_id	INTEGER	Original OLTP ID
full_name	VARCHAR(100)	First + last name
email	VARCHAR(100)	Staff email
store_id	INTEGER	Store where the staff member works
active_status	VARCHAR(10)	Active or Inactive

3.4 Schema Type

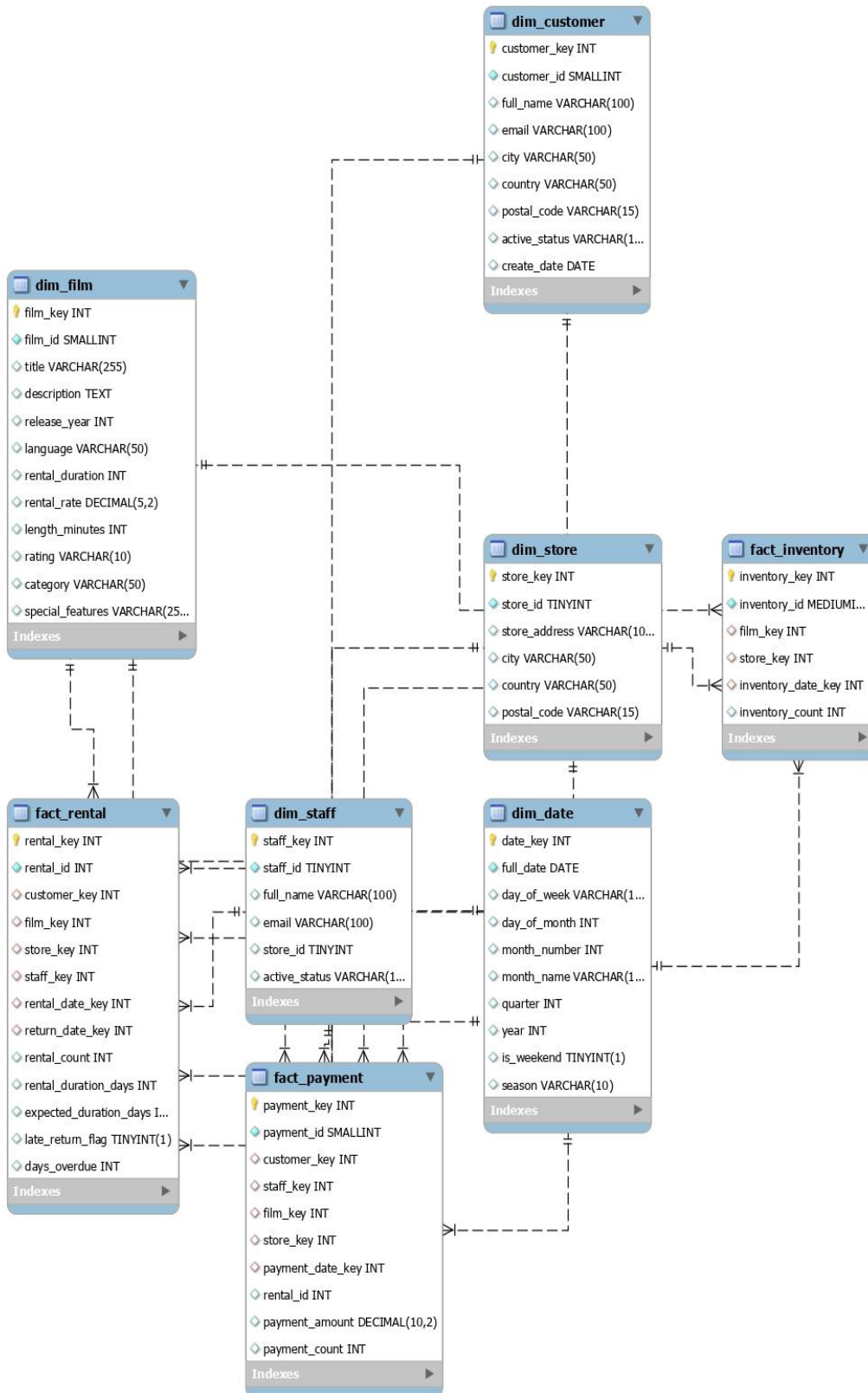
The design uses a Star Schema. Reasons:

- Fast query performance — dimension tables are not normalized further.
- Lookup tables in the OLTP (language, category, city, country) are flattened directly into dimension tables without data quality loss.

Note on film-category: In OLTP, a film can belong to multiple categories (many-to-many via film_category). In the warehouse, the primary category is stored directly in Dim_Film to keep the Star Schema simple. This is a deliberate trade-off favoring query speed over full flexibility.

Section 4: Dimensional Model Diagram

The diagram below shows the Star Schema used for the Movie Rental Data Warehouse. The warehouse contains three fact tables: Fact_Rental, Fact_Payment, and Fact_Inventory. These fact tables are connected to shared dimension tables including Dim_Date, Dim_Customer, Dim_Film, Dim_Store, and Dim_Staff. The design supports rental analysis, revenue analysis, inventory availability analysis, customer behavior analysis, store performance analysis, and time-based reporting. The Star Schema structure simplifies analytical queries and improves reporting performance.



Section 5: ETL Design

ETL moves data from the OLTP movie rental database into the warehouse. The three phases are Extract, Transform, and Load.

5.1 Extract

The following OLTP tables are extracted. Each is needed for a specific purpose:

OLTP Table	Used To Build	Reason
rental	Fact_Rental	Core rental events: dates, customer, staff, inventory references
payment	Fact_Payment	Payment events: amount, date, customer, staff, rental reference
customer	Dim_Customer	Customer personal information
address	Dim_Customer, Dim_Store, Dim_Staff	Physical address for customers, stores, staff
city	Dim_Customer, Dim_Store, Dim_Staff	City names for location analysis
country	Dim_Customer, Dim_Store, Dim_Staff	Country names for geographic reporting
film	Dim_Film	Film details: title, rating, rental duration and rate
language	Dim_Film	Language name for the film language attribute
category	Dim_Film	Film category names
film_category	Dim_Film	Links films to categories (many-to-many junction)
inventory	Fact_Rental, Fact_Payment, Fact_Inventory	Maps rentals/payments to films and stores, and provides physical inventory copies
store	Dim_Store	Store location information
staff	Dim_Staff	Staff name, email, store assignment

5.2 Transform

Dim_Date

- Generated programmatically — not extracted from OLTP.
- One row per date in the required range (e.g. 2000-01-01 to 2030-12-31).
- Derive: day_of_week, month_name, quarter, year, is_weekend, season.
- date_key = YYYYMMDD integer (e.g. 20240215).

Dim_Customer

- JOIN: customer → address → city → country.
- Concatenate: full_name = first_name + ' ' + last_name.
- Map active field: 1 = 'Active', 0 = 'Inactive'.
- TRIM and standardize city/country text.
- Replace NULL email or address with 'Unknown'.
- Generate surrogate key: customer_key (auto-increment).

Dim_Film

- JOIN: film → language; film → film_category → category.
- For films with multiple categories: select the primary category (first match).
- Validate rating: must be G, PG, PG-13, R, or NC-17. Set to 'Unknown' otherwise.
- Clean description text (trim whitespace).
- Generate surrogate key: film_key.

Dim_Store

- JOIN: store → address → city → country.
- Combine address fields into a clean store_address string.
- Generate surrogate key: store_key.

Dim_Staff

- JOIN: staff → address → city → country → store (for store location context).
- Concatenate full_name from first_name and last_name.
- Map active field to 'Active'/'Inactive'.
- Generate surrogate key: staff_key.

Fact_Rental

- JOIN: rental → inventory (to get film_id and store_id).
- Map rental_date → date_key; return_date → return_date_key via Dim_Date lookup.
- Handle NULL return_date carefully when the film has not been returned yet.
- Replace OLTP IDs with warehouse surrogate keys for customer, film, store, staff.
- Calculate rental_duration_days = DATEDIFF(return_date, rental_date).
- Calculate late_return_flag = 1 if rental_duration_days > expected_duration_days.
- Calculate days_overdue = MAX(0, rental_duration_days - expected_duration_days).

Fact_Payment

- Map payment_date → payment_date_key via Dim_Date.
- Replace customer_id and staff_id with warehouse surrogate keys.
- Use rental_id to join payment → rental → inventory in order to derive film_key and store_key.
- Store rental_id as the original rental reference from the OLTP system.
- Validate: payment_amount must be > 0 and not NULL.
- Remove duplicate payment_id records.

Fact_Inventory

- JOIN: inventory → film and inventory → store.
- • Replace film_id and store_id with warehouse surrogate keys.

- Map inventory.last_update to inventory_date_key.
- Store inventory_id as the original inventory reference.
- Set inventory_count = 1 for each physical copy.

5.3 Load

Loading order respects referential integrity — dimension tables are loaded before fact tables:

1. Dim_Date
2. Dim_Customer
3. Dim_Film
4. Dim_Store
5. Dim_Staff
6. Fact_Inventory
7. Fact_Rental
8. Fact_Payment

Key loading rules

- Surrogate keys are generated by the ETL process using auto-increment integers.
- Fact table rows reference surrogate keys, not OLTP natural IDs.
- Initial load: INSERT all transformed records.
- Incremental loads: insert new records and update changed records.
- Verify all surrogate key references exist in their dimension before loading fact rows.

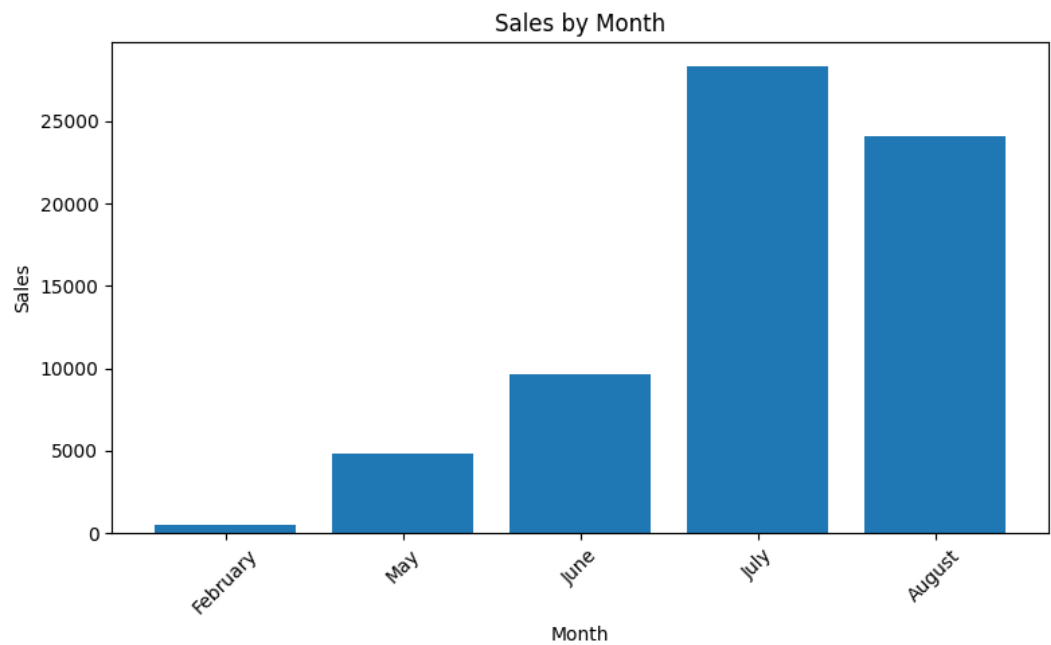
Section 6: Data Quality Considerations

Rule ID	Rule Description	Action if Violated
DQ-01	rental.customer_id must exist in the customer table	Reject record, log error
DQ-02	payment.amount must be > 0	Reject record, log error
DQ-03	return_date >= rental_date when not NULL	Flag as error, set return_date to NULL
DQ-04	film.rating must be G, PG, PG-13, R, or NC-17	Set to 'Unknown'
DQ-05	All FK columns in fact tables must reference existing dimension rows	Reject fact record, log orphan key
DQ-06	No duplicate payment_id values in Fact_Payment	Keep first, discard duplicates
DQ-07	inventory.film_id and inventory.store_id must exist	Reject inventory record, log orphan key

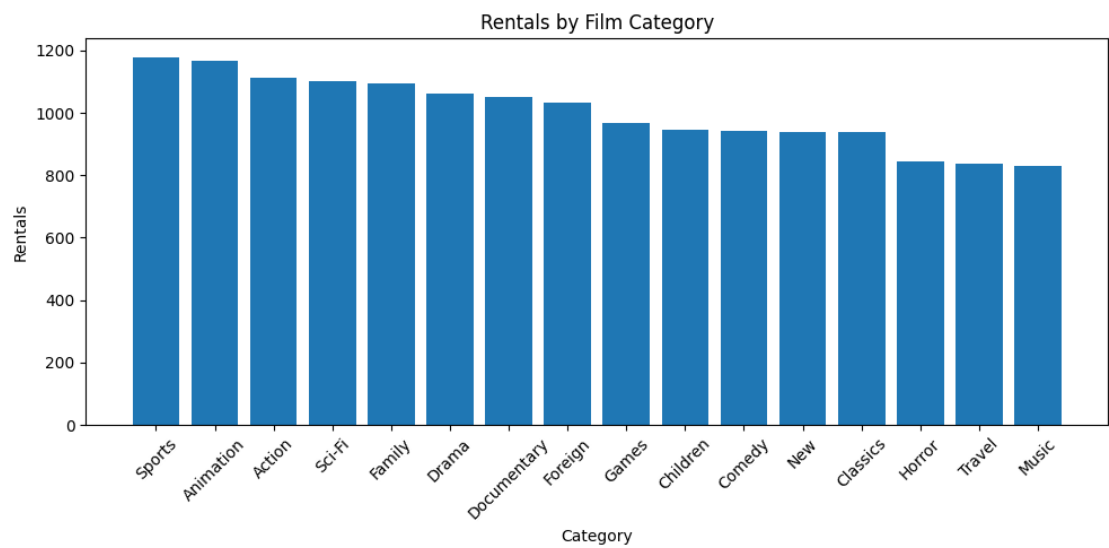
After each ETL run, a summary report should be generated: total records extracted, records passed, records rejected, and rejection reasons.

Section 7: Sample Analytical Visualizations

1) Revenue by Month



2) Rentals by Film Category



3) Inventory Copies by Store



These visualizations were generated from the implemented analytical queries in the Jupyter Notebook.

Section 8: Conclusion

This report presented a proposed data warehouse design for a movie rental business. The design uses a Star Schema with three fact tables: Fact_Rental, Fact_Payment, and Fact_Inventory, and five dimension tables: Dim_Date, Dim_Customer, Dim_Film, Dim_Store, and Dim_Staff.

The proposed model supports common business analysis such as identifying popular films, tracking customer activity, comparing store performance, monitoring payments, and studying trends over time. The ETL process explains how data can be extracted from the OLTP system, cleaned, transformed, and loaded into the warehouse.